

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	MANDLA VISWATEJA	Reg. No.:	192311399
Section / Batch:	CSE/2023	Date:	18/07/2026

Instructions to Students

- Answer ALL questions. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1 Q2,Q3	40 60 (2X30)
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

Design and Analysis of Algorithms (CSA06) — CO2 Assessment
Full Answers for Q1–Q5 (5 × 20 = 100 Marks)

Q1. Selection Sort – Analytical Problem (20 Marks)

Question: Apply Selection Sort on the dataset [64, 25, 12, 22, 11]. Show each pass, count comparisons, analyze time complexity, and verify the sorted output.

1. Problem Interpretation

We must sort the array in ascending order using the Selection Sort brute-force technique. Selection Sort repeatedly selects the minimum element from the unsorted portion and places it at the correct position.

2. Why Selection Sort?

- Simple and easy to implement.
- Performs a fixed number of comparisons regardless of input order.
- Suitable for small datasets.

3. Step-by-Step Execution

Initial array: [64, 25, 12, 22, 11]

Pass	Minimum found	Array after pass
1	11	[11, 25, 12, 22, 64]
2	12	[11, 12, 25, 22, 64]
3	22	[11, 12, 22, 25, 64]
4	25	[11, 12, 22, 25, 64]

4. Comparison Count

For $n = 5$:

- Pass 1: 4 comparisons
- Pass 2: 3 comparisons
- Pass 3: 2 comparisons
- Pass 4: 1 comparison

Total comparisons = $4 + 3 + 2 + 1 = 10$

5. Swap Count

- Pass 1: swap 64 and 11
- Pass 2: swap 25 and 12
- Pass 3: swap 25 and 22
- Pass 4: no effective swap

Total swaps = 3

6. Time Complexity

- Best case: $O(n^2)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

7. Correctness Verification

Final array: [11, 12, 22, 25, 64]. Each element is less than or equal to the next element, so the array is correctly sorted.

8. Interpretation

Selection Sort made 10 comparisons and 3 swaps. It minimizes swaps compared to Bubble Sort, but its quadratic time complexity makes it inefficient for large datasets.

Q2. Closest Pair – Brute Force Analysis (20 Marks)

Question: For points $\{(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)\}$, find the closest pair using the brute-force method.

1. Problem Interpretation

We must compute the Euclidean distance between every pair of points and identify the pair with the minimum distance.

2. Distance Formula

$$d((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. Pairwise Distances

Pair	Distance
(2,3) – (12,30)	$\sqrt{829} \approx 28.79$
(2,3) – (40,50)	$\sqrt{3653} \approx 60.44$
(2,3) – (5,1)	$\sqrt{13} \approx 3.61$
(2,3) – (12,10)	$\sqrt{149} \approx 12.21$
(2,3) – (3,4)	$\sqrt{2} \approx 1.41$
(12,30) – (40,50)	$\sqrt{1184} \approx 34.41$
(12,30) – (5,1)	$\sqrt{890} \approx 29.83$
(12,30) – (12,10)	20
(12,30) – (3,4)	$\sqrt{757} \approx 27.51$
(40,50) – (5,1)	$\sqrt{3626} \approx 60.22$
(40,50) – (12,10)	$\sqrt{2384} \approx 48.83$
(40,50) – (3,4)	$\sqrt{3485} \approx 59.03$
(5,1) – (12,10)	$\sqrt{130} \approx 11.40$
(5,1) – (3,4)	$\sqrt{13} \approx 3.61$
(12,10) – (3,4)	$\sqrt{117} \approx 10.82$

4. Closest Pair

The minimum distance is $\sqrt{2} \approx 1.41$ between **(2,3)** and **(3,4)**.

5. Comparison Count

For $n = 6$: Number of pairwise comparisons = $n(n-1)/2 = 6 \times 5/2 = 15$.

6. Time Complexity

- Distance computations: 15
- Overall complexity: $O(n^2)$

7. Correctness Verification

All 15 pairs were examined; no distance smaller than $\sqrt{2}$ was found.

8. Real-World Significance

Closest-pair algorithms are used in GPS navigation, emergency-center selection, clustering, and collision detection.

Q3. Convex Hull – Brute Force Approach (20 Marks)

Question: Apply the brute-force Convex Hull algorithm on $\{(0,3), (2,2), (1,1), (2,1), (3,0), (0,0), (3,3)\}$.

1. Problem Interpretation

The convex hull is the smallest convex polygon that contains all given points.

2. Extreme Points

The outermost points are:

- (0,0)
- (3,0)
- (3,3)
- (0,3)

The points (2,2), (1,1), and (2,1) lie inside the boundary.

3. Hull Edge Validation

For each candidate edge, all other points must lie on the same side of the line. Valid hull edges:

- (0,0) $\rightarrow$ (3,0)
- (3,0) $\rightarrow$ (3,3)
- (3,3) $\rightarrow$ (0,3)
- (0,3) $\rightarrow$ (0,0)

4. Total Comparisons

- Number of point pairs = $7C2 = 21$
- For each pair, up to 5 remaining points are checked.

Maximum orientation checks = $21 \times 5 = 105$

5. Time Complexity

Brute-force Convex Hull checks every pair against all other points. **Time complexity = $O(n^3)$**

6. Final Convex Hull

Traversing counterclockwise: **(0,0) $\rightarrow$ (3,0) $\rightarrow$ (3,3) $\rightarrow$ (0,3) $\rightarrow$ (0,0)**

7. Practical Significance

Convex hulls are used in computer graphics, GIS mapping, robotics path planning, and image processing.

Q4. Searching Techniques – Comparative Analysis (20 Marks)

Question: Analyze the performance of Linear Search on unsorted data and Binary Search on sorted data.

1. Problem Interpretation

Compare Linear Search (for unsorted data) and Binary Search (for sorted data).

2. Working Principle

Linear Search: Check elements sequentially from the beginning until the target is found or the list ends.

Binary Search: Repeatedly compare the target with the middle element and halve the search interval.

3. Example

Dataset: [7, 14, 21, 28, 35, 42, 49] Target: 35

Linear Search

Comparisons: 7 (1) → 14 (2) → 21 (3) → 28 (4) → 35 (5)

Total = 5 comparisons

Binary Search

- Mid = 28 (1) → target is larger.
- Search right half [35, 42, 49].
- Mid = 42 (2) → target is smaller.
- Search left half [35].
- Mid = 35 (3) → found.

Total = 3 comparisons

4. Time Complexity Comparison

Case	Linear Search	Binary Search
Best	$O(1)$	$O(1)$
Average	$O(n)$	$O(\log n)$
Worst	$O(n)$	$O(\log n)$

5. When to Use

Situation	Preferred Method
Unsorted data	Linear Search
Small dataset	Linear Search
Sorted large dataset	Binary Search
Frequent searches	Binary Search

6. Interpretation

Binary Search is significantly more efficient for large sorted datasets because it reduces the search space by half at each step.

Q5. Bubble Sort – Analytical Problem (20 Marks)

Question: Apply Bubble Sort on [45, 12, 78, 34, 23]. Show each pass, count comparisons and swaps, analyze complexity, and verify the result.

1. Problem Interpretation

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

2. Step-by-Step Execution

Initial array: [45, 12, 78, 34, 23]

Pass 1

- $45 > 12 \rightarrow \text{swap} \rightarrow [12, 45, 78, 34, 23]$
- $45 < 78 \rightarrow \text{no swap}$
- $78 > 34 \rightarrow \text{swap} \rightarrow [12, 45, 34, 78, 23]$
- $78 > 23 \rightarrow \text{swap} \rightarrow [12, 45, 34, 23, 78]$

Pass 2

- $12 < 45 \rightarrow \text{no swap}$
- $45 > 34 \rightarrow \text{swap} \rightarrow [12, 34, 45, 23, 78]$
- $45 > 23 \rightarrow \text{swap} \rightarrow [12, 34, 23, 45, 78]$

Pass 3

- $12 < 34 \rightarrow \text{no swap}$
- $34 > 23 \rightarrow \text{swap} \rightarrow [12, 23, 34, 45, 78]$

Pass 4

- $12 < 23 \rightarrow \text{no swap}$

Final sorted array: [12, 23, 34, 45, 78]

3. Comparison Count

- Pass 1: 4
- Pass 2: 3
- Pass 3: 2
- Pass 4: 1

Total comparisons = 10

4. Swap Count

- Pass 1: 3 swaps
- Pass 2: 2 swaps
- Pass 3: 1 swap
- Pass 4: 0 swaps

Total swaps = 6

5. Time Complexity

- Best case (optimized): $O(n)$

- Average case: $O(n^2)$
- Worst case: $O(n^2)$

6. Correctness Verification

Final array [12, 23, 34, 45, 78] is in ascending order.

7. Interpretation

Bubble Sort required 10 comparisons and 6 swaps. It is easy to understand but generally less efficient than Selection Sort because it performs more swaps.

Conclusion

These five solutions together cover 100 marks (5×20 marks) and address all major CO2 outcomes: Selection Sort, Bubble Sort, Linear vs Binary Search, Closest Pair, and Convex Hull. Each answer includes interpretation, algorithm justification, step-by-step execution, comparison counts, complexity analysis, correctness verification, and practical significance, matching the requirements of the assessment.